

Updating an Item in Azure Cosmos DB Using the .NET SDK

Summary:-

In this lab, you will update an existing document in an Azure Cosmos DB container using the Azure Cosmos DB .NET SDK. You will retrieve an item using its **Item ID** and **Partition Key**, modify one of its properties, and replace the existing document in the container.

Let's Start:-

After completing this lab, you will be able to:

- Read an existing document from Azure Cosmos DB.
- Modify the properties of a retrieved document.
- Update an item using the ReplaceItemAsync() method.
- Verify the updated document in Azure Cosmos DB.

Lab Steps:-

Step 1 – Prepare the Azure Cosmos DB .NET Project

- **Reference:** This lab uses the .NET console application, Azure Cosmos DB account, database, and container created in the previous lab.
- For that **create** Azure Cosmos DB account by selecting **Azure Cosmos DB for NoSQL** from the recommended APIs.
- Configure:
Workload type: Learning, **Region:** EAST US/ EAST US 2, **Capacity Mode:** Provisioned Throughput
- Enable **Apply Free Tier Discount**. Click **Next**.
- Ensure Geo-Redundancy and Multi-region Writes is **Disabled**. Click **Next**.
- Select **Enable access from all networks**. Click on **Review + Create** to create.
- From the left pane, select **Keys**.
- Copy: **URI** and **Primary Key**
- Open **Visual Studio Code**. Create a new project folder.
- Press **Ctrl + Shift + P** to open the **Command Palette**.
- Select **Show and Run Commands** > **.NET: New Project** > **Console App**(Rename it as cosmosdb-app) > **Default Directory** > **.slnx** > **Create Project**.
- **Open** the integrated terminal. Run:

```
cd .\cosmosdb-app\  
dotnet add package Microsoft.Azure.Cosmos  
dotnet add package Newtonsoft.Json
```
- Create a new class file named **Order.cs**.
- Open the following project files:

Program.cs

Order.cs

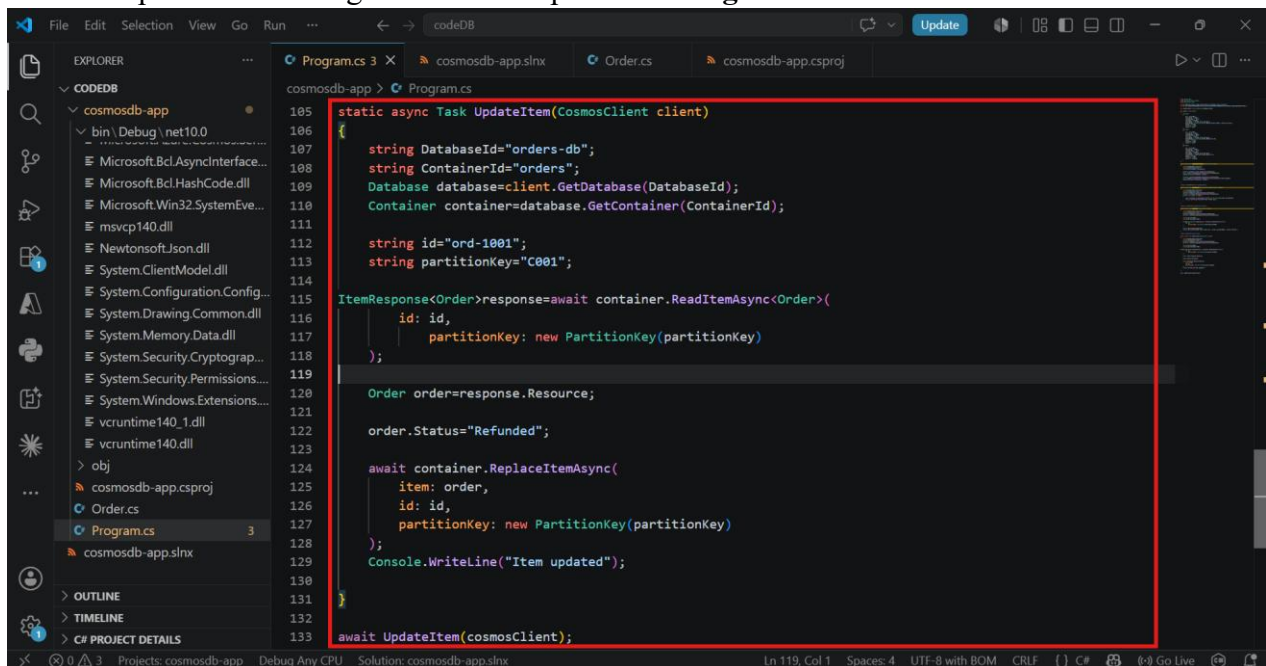
cosmosdb-app.csproj

cosmosdb-app.slnx

- Replace the existing content with the **provided files**.
- In **Program.cs**, **Replace** Endpoint with **URI** and Key with **Primary Key**.
- **Save** all the files. **Run**:
dotnet run
dotnet build
- This step creates a .NET console application and prepares it to insert items into the Azure Cosmos DB container.

Step 2 – Add the Update Item Code

- Open **Program.cs**.
- Replace the existing code with the provided **Program.cs** file.

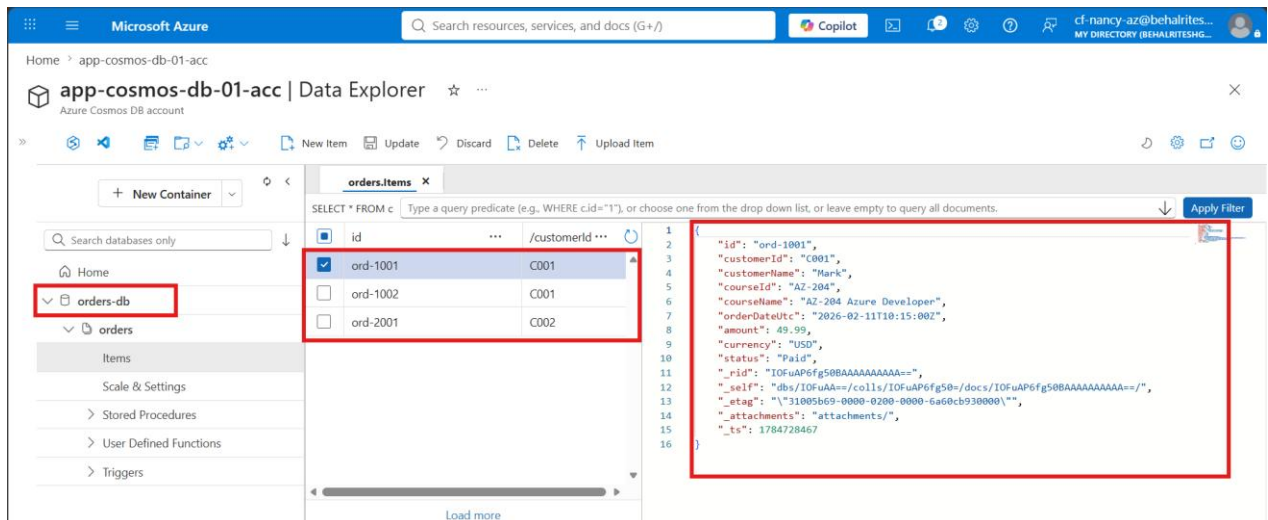


```
105 static async Task UpdateItem(CosmosClient client)
106 {
107     string DatabaseId="orders-db";
108     string ContainerId="orders";
109     Database database=client.GetDatabase(DatabaseId);
110     Container container=database.GetContainer(ContainerId);
111
112     string id="ord-1001";
113     string partitionKey="C001";
114
115     ItemResponse<Order>response=await container.ReadItemAsync<Order>({
116         id: id,
117         partitionKey: new PartitionKey(partitionKey)
118     });
119
120     Order order=response.Resource;
121
122     order.Status="Refunded";
123
124     await container.ReplaceItemAsync(
125         item: order,
126         id: id,
127         partitionKey: new PartitionKey(partitionKey)
128     );
129     Console.WriteLine("Item updated");
130 }
131
132 await UpdateItem(cosmosClient);
133
```

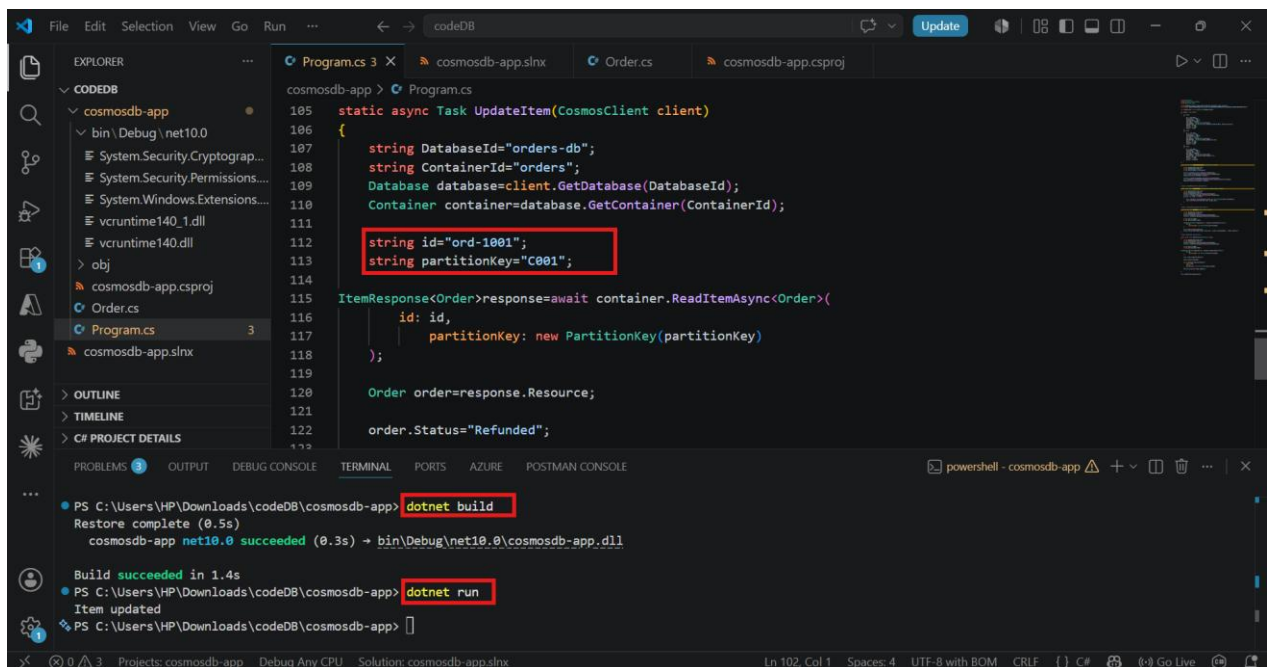
- **Save** the file.
- This step adds the code required to update an existing document in Azure Cosmos DB.

Step 3 – Update the Item

- In the code, specify:
 - **Item ID:** ord-1001
 - **Partition Key:** C001
- Observe that the **Status** field is currently set to **"Paid"** before running the application.



- **Build and run the application.** Run:
dotnet build
dotnet run



- This step reads the existing document, updates its status, and replaces the document in the container.

Step 4 – View the Retrieved Item

- Open the **Azure Portal**. Navigate to **Azure Cosmos DB > Data Explorer**.
- Open the **orders** container.
- Select the updated document (ord-1001).

The screenshot shows the Microsoft Azure Data Explorer interface. On the left, the 'Data Explorer' sidebar is visible, with 'orders' selected under 'orders-db'. The main pane shows a table of documents with columns 'id' and 'order'. The document 'ord-1001' is selected and highlighted. The right pane displays the JSON content of the selected document, with the 'status' field highlighted in red, showing 'Refunded'.

```

1 {
2   "id": "ord-1001",
3   "customerId": "C001",
4   "customerName": "Mark",
5   "courseId": "AZ-204",
6   "courseName": "AZ-204 Azure Developer",
7   "orderDateUtc": "2026-02-11T10:15:00Z",
8   "amount": 49.99,
9   "currency": "USD",
10  "status": "Refunded",
11  "_rid": "x6R2APar1uUBAAAAAAAAA==",
12  "_self": "dbs/x6R2AA==/colls/x6R2APar1uUBAAAAA",
13  "_etag": "\"c600473f-0000-0200-0000-6a6337160000\"",
14  "attachments": "attachments/",
15  "_ts": 1784887862
16 }

```

- Verify that the **Status** field has been updated successfully.
- This step confirms that the document has been updated successfully in Azure Cosmos DB.

Verification

Verify the following:

- The application runs successfully without errors.
- The specified document is retrieved correctly.
- The **Status** field is updated successfully.
- The updated document is visible in the Azure Cosmos DB container.
- The changes are saved successfully using the **ReplaceItemAsync()** method.